COMPSCI 5100 ML & AI for Data Science

Fani Deligianni

fani.deligianni@glasgow.ac.uk

https://fd301.github.io/index.html

Classification: Part I - Nearest Neighbour

Some slides are adapted from previous lectures: Dr. Yuan, Dr. Guha, Dr. Gooya

Learning Objectives

- To be able to formulate a problem to a classification task
- To understand the components of a classifier
- To be able to evaluate a classifier's performance

ML paradigms

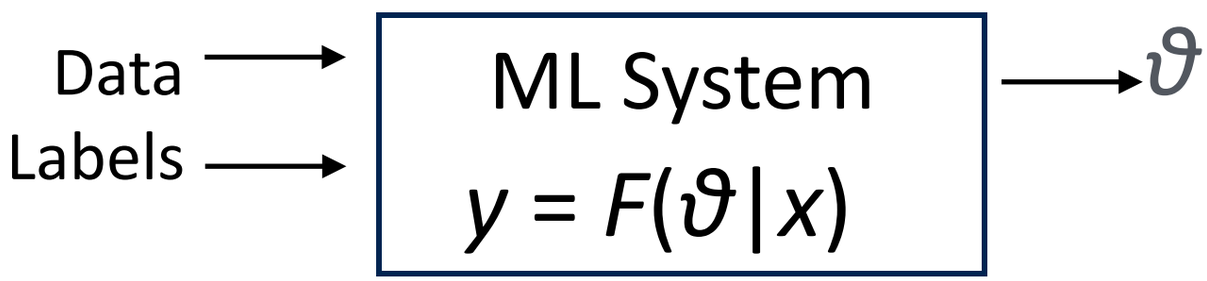
[AI ≠ ML, but in today's world ML and AI are often synonymous]

Machine Learning paradigms

- **Supervised**
  - need labelled training data
- **Semi-supervised**
  - some labelled training data
- **Unsupervised/ Self-supervised**
  - no labelled training data needed

Supervised learning paradigm

- **Observe** a set of examples:
- Training data (measurements of any kind)= x
- Lables =y
- **Model** the relationship between data and labels
- **Predict** the label for a new data (test data)



$$y_{test} = F(x_{test}|\vartheta)$$

Classification vs. Regression

Regression

- Lables = $\mathbf{y}$ is continuous
- For example, y predicts the price of houses in Glasgow
- Regression models are discriminative models

Classification

- Lables = $\mathbf{y}$ is discrete
- For example, y identifies an image as: *dog, cat, mouse* (class)
- As the number of classes increase, difficulty task increases
- Can be either discriminative models or generative models

Can we transform a regression task to a classification task?

Components of a classification system

1. Labeled data for training

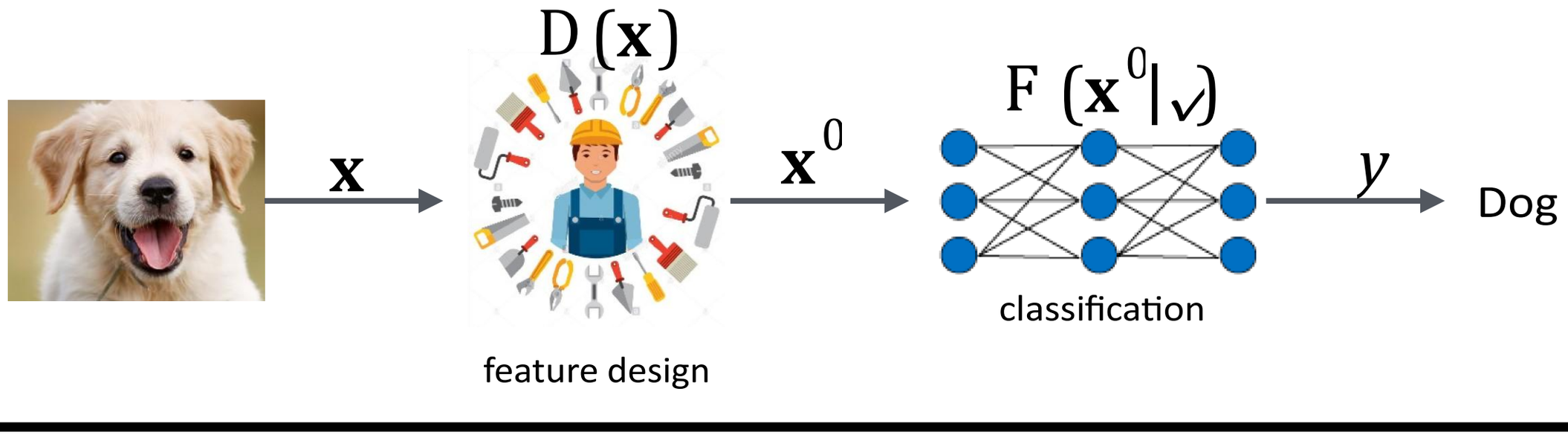
2. Features (hand crafted for traditional ML, learned in DL)

3. Model

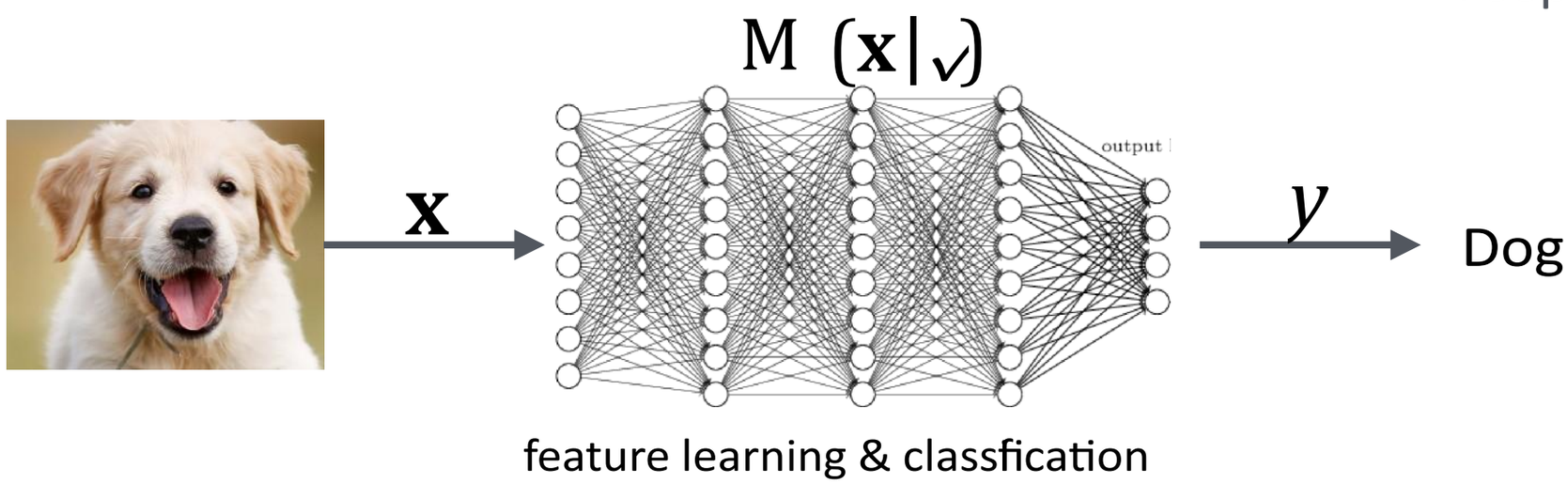
4. Evaluation metrics

Components of a classification system

Traditional machine learning

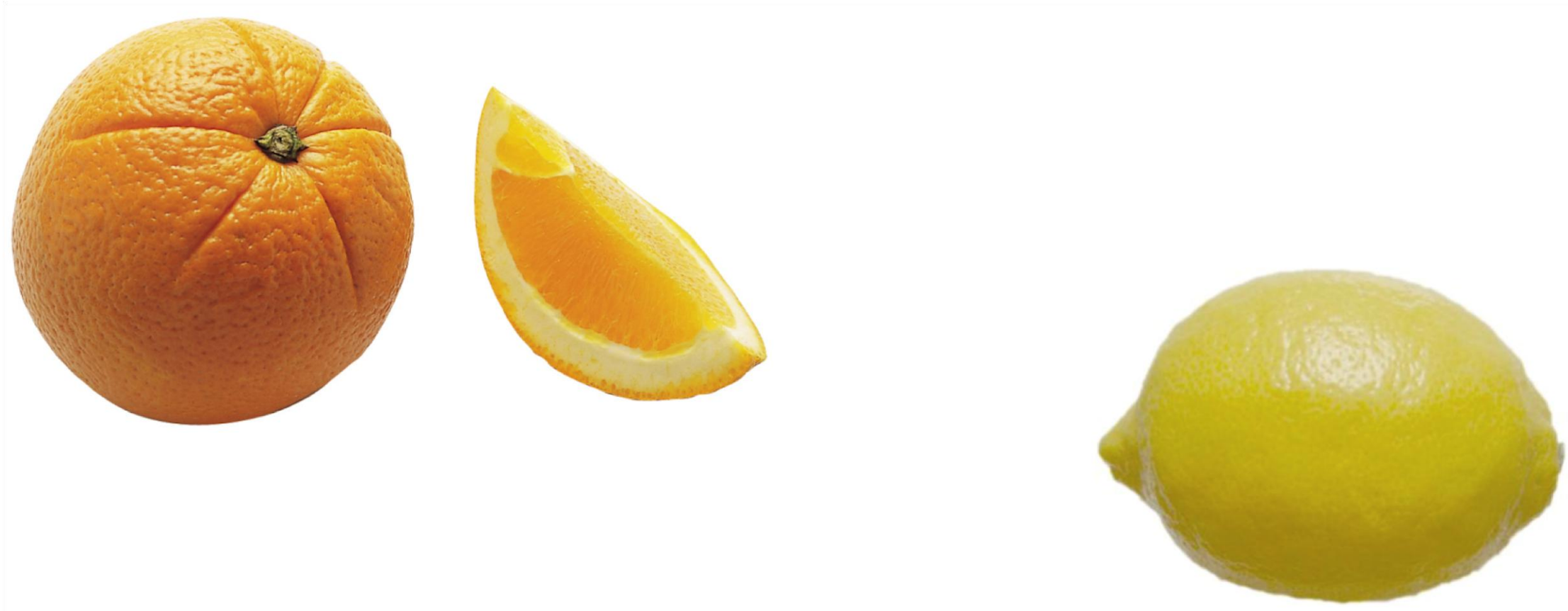


Deep learning



Example

Task: Build a classification system to identify oranges and lemons



(not necessarily using images)

Example

Task: Build a classification system to identify oranges and lemons

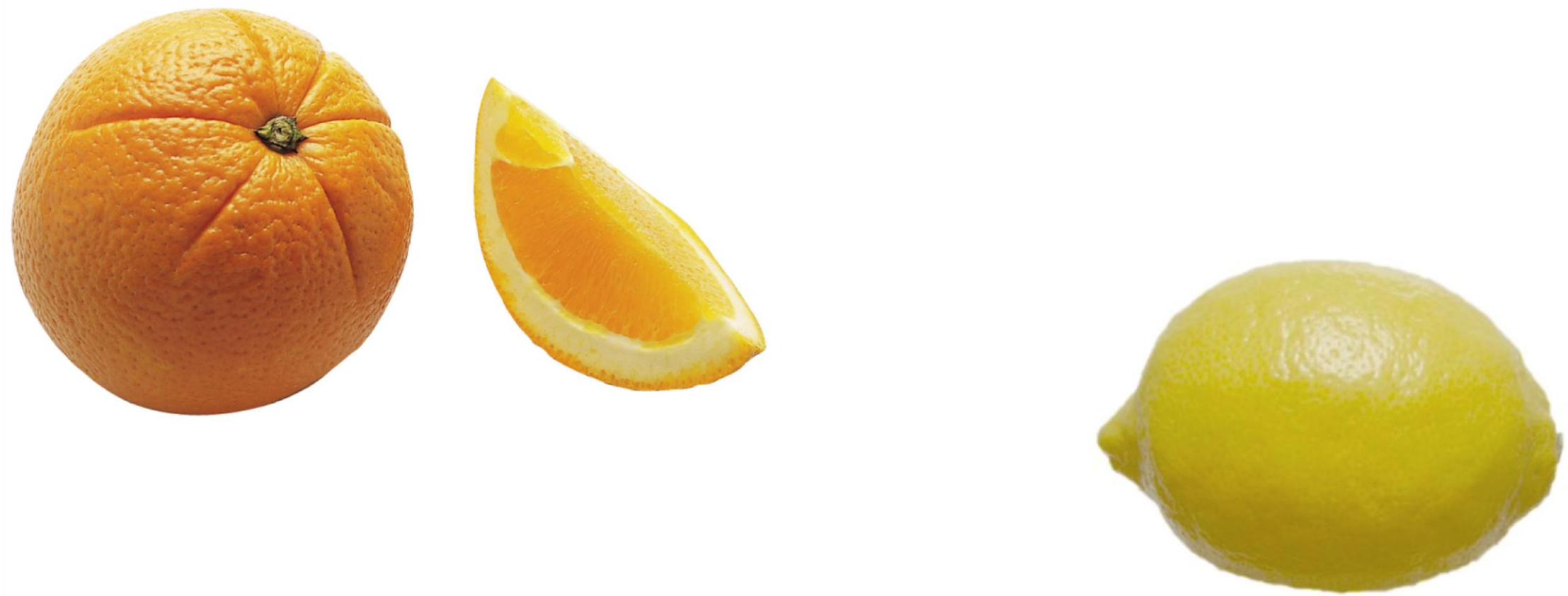


Step 1: Decide which attributes can you measure

- weight
- shape
- colour (images)

Example

Task: Build a classification system to identify oranges and lemons



Step 2: Collect data to create training set

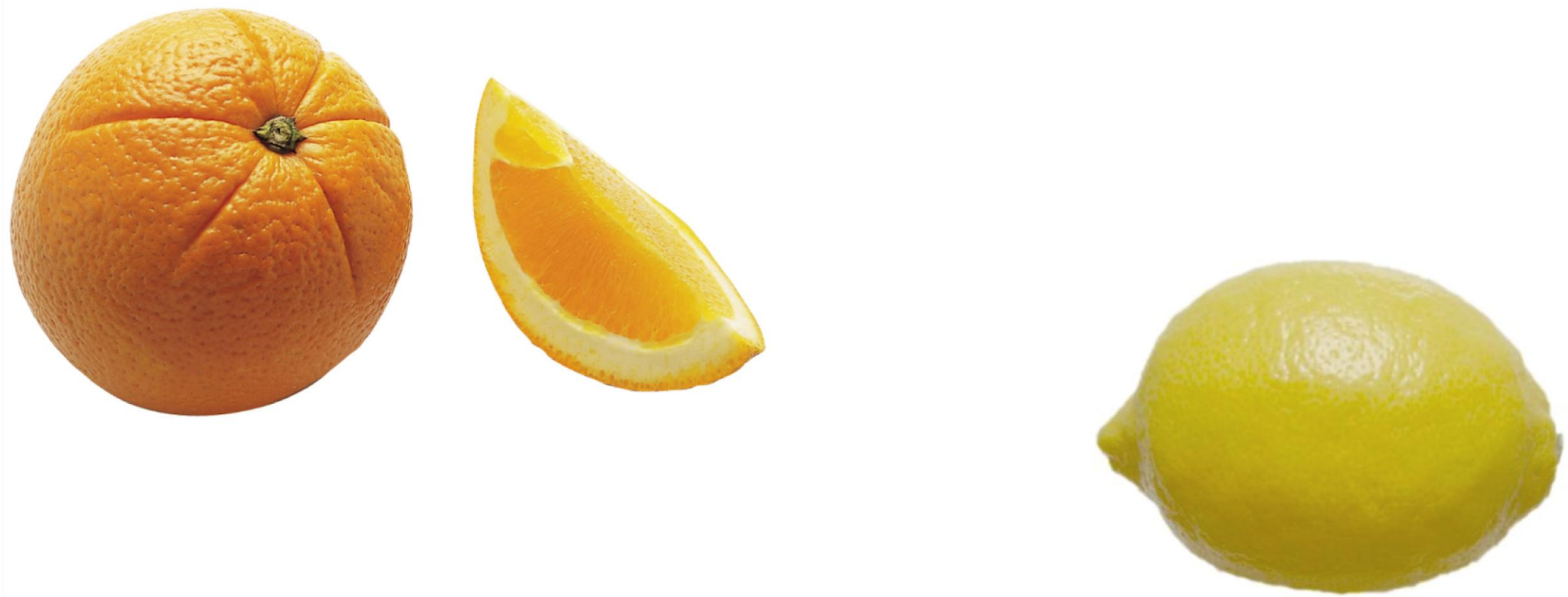
(Ideal) Training data

- **ML models can not extract features automatically**
- Need a (large) set of training data
- Balanced samples across classes
- Not noisy (accurate measurement)
- Varied examples for each class



Example

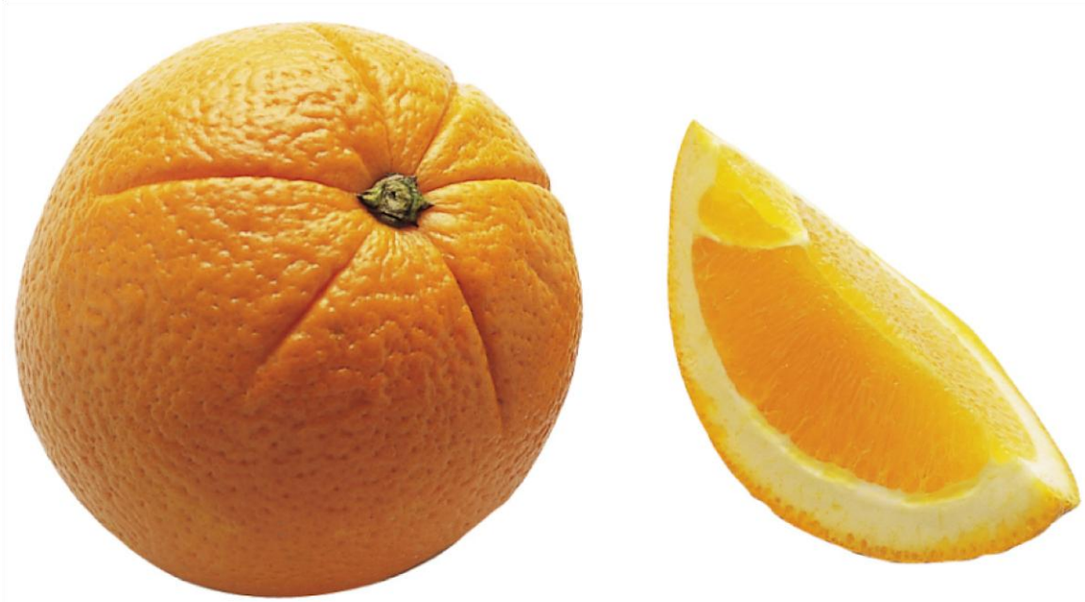
Task: Build a classification system to identify oranges and lemons



Step 3: Extract features

Example

Task: Build a classification system to identify oranges and lemons

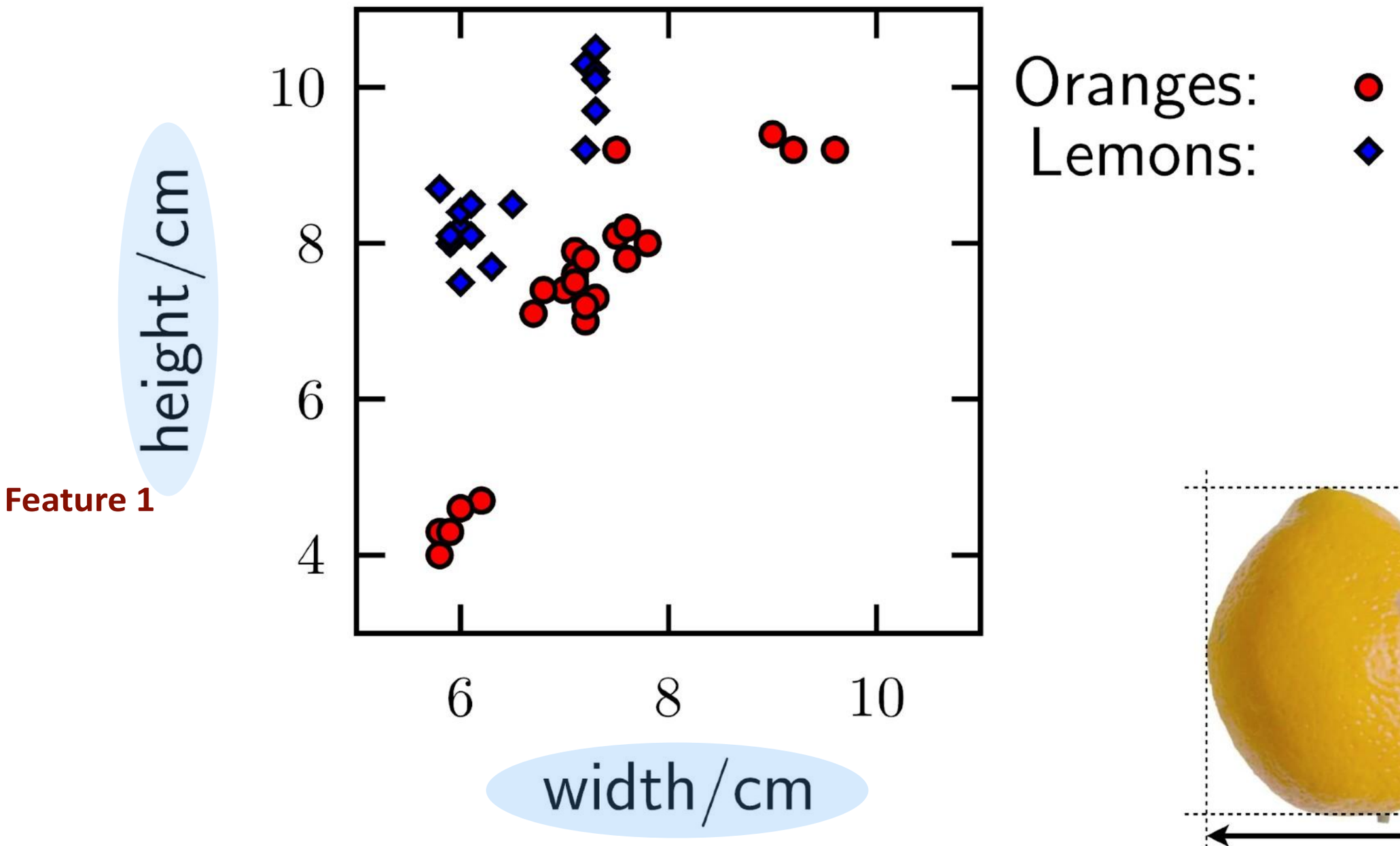


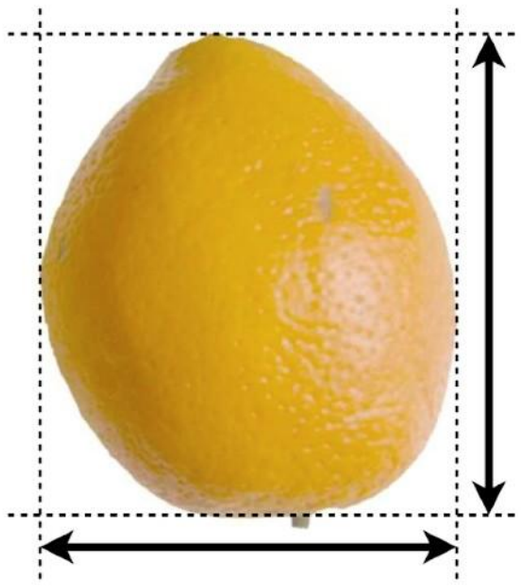


(not necessarily using images)

Step 4: Build ML Model (classifier)

Training data





Notations

- N training samples (raw data or features): $\mathbf{x}_1, \mathbf{x}_2, \cdots \mathbf{x}_N$
- Each sample associated with a label: $y_1, y_2, \cdots y_N$
- Binary classification: $y_n \in \{0,1\}$ where $n = 1,2 \cdots N$
- Multiclass classification: $y_n \in \{1,2, \cdots C\}$
- Test sample: $\mathbf{x}_{new}$
- Task: Assign a label y_{new} to $\mathbf{x}_{new}$ where $y_{new} \in \{1,2 \ldots C\}$

  (multi-class) or $y_n \in \{0,1\}$ (if binary)

Notations

- Instead of a discrete number, classifiers may output a probability score, i.e., a number between 0 and 1, corresponding to each class.

- Binary classification example
- $P(y_{new} = 0 \mid x_{new}, \Theta) = 0.8$
- $P(y_{new} = 1 \mid x_{new}, \Theta) = 0.2$

- These values can also be interpreted as 'how sure our model is'

Types of models

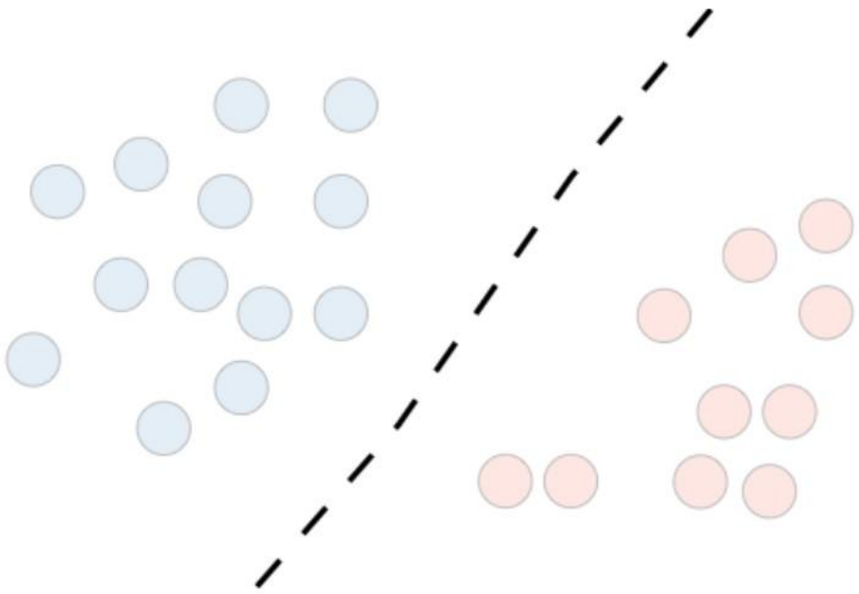
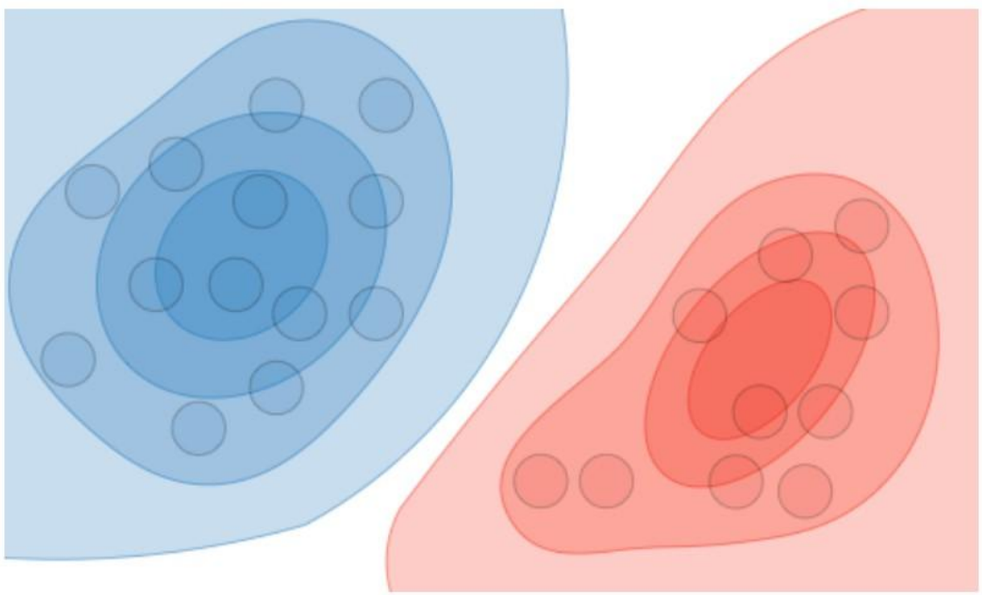
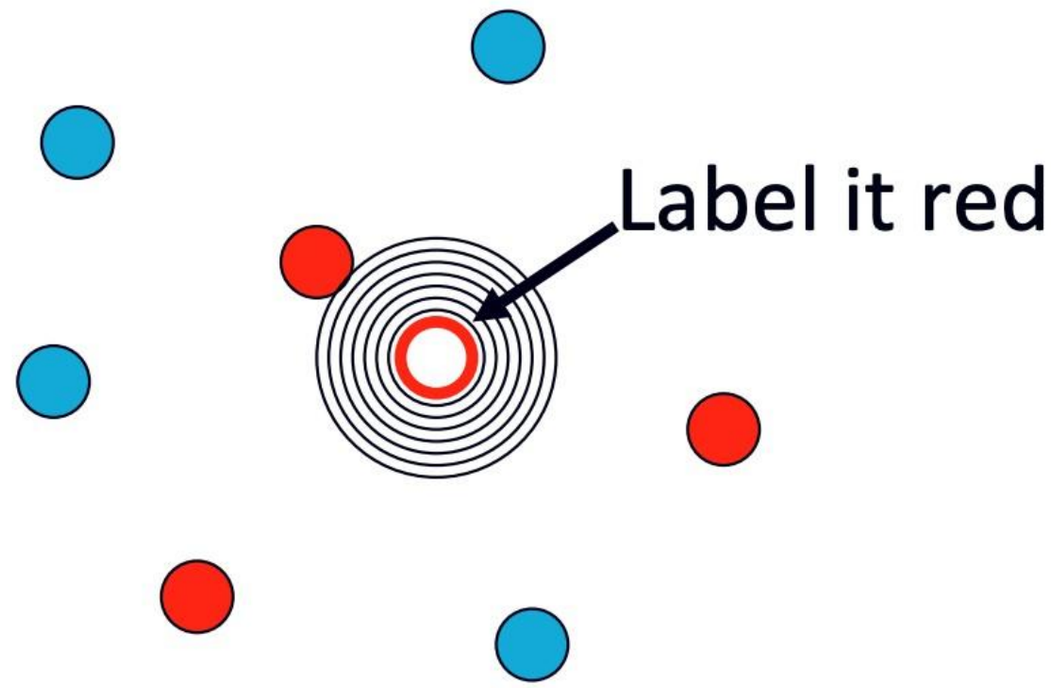
	Discriminative model	Generative model
Goal	Directly estimate $P(y\|x)$	Estimate $P(x\|y)$ to then deduce $P(y\|x)$
What's learned	Decision boundary	Probability distributions of the data
Illustration		
Examples	Regressions, SVMs	GDA, Naive Bayes

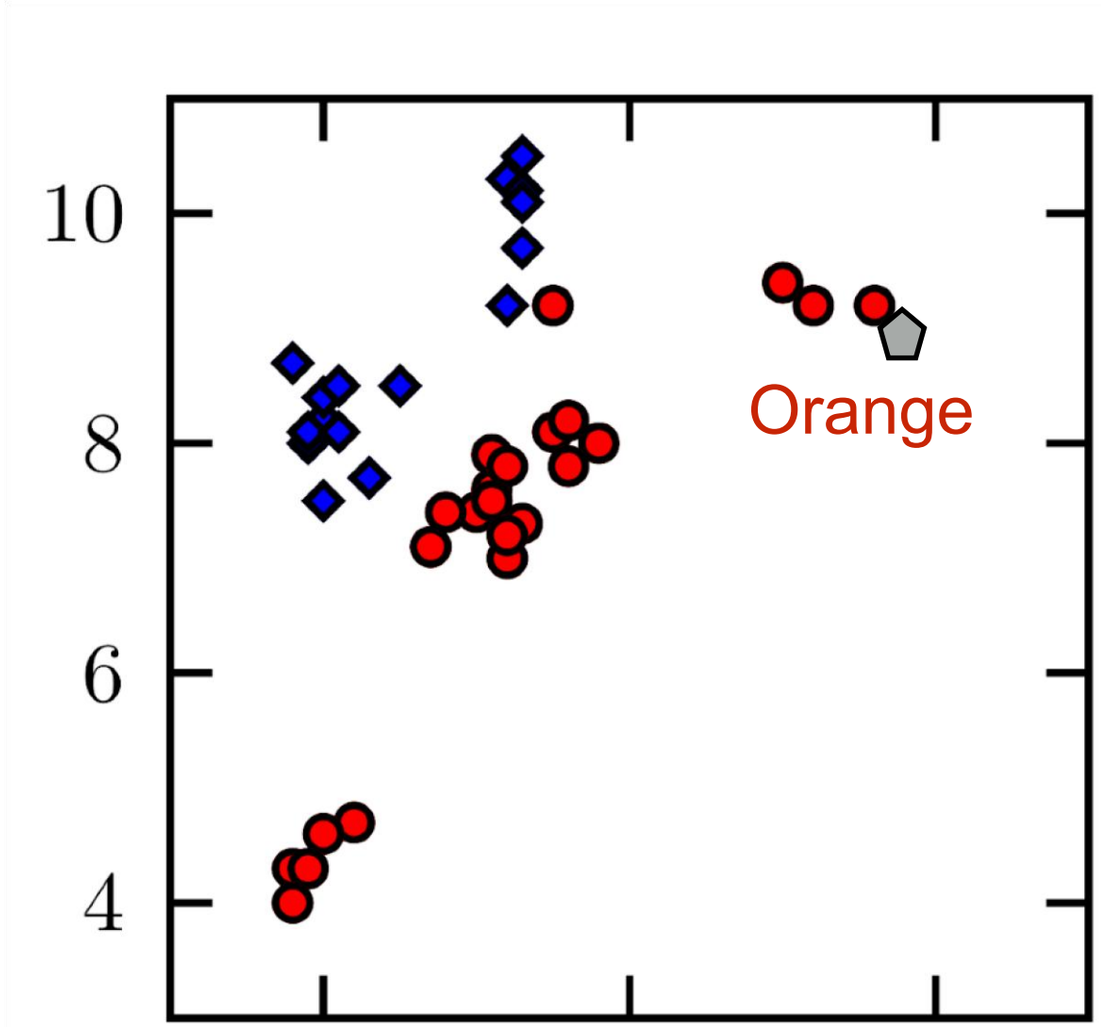
Image from: https://stanford.edu/~shervine/teaching/cs-229/

Nearest Neighbour

- Simplest of all classifiers: **1-Nearest Neighbour**
- **Simple idea:** Label a new sample the same as its closest data point



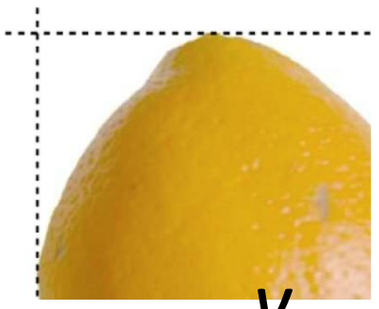
1-Nearest Neighbour



Oranges:

Lemons:

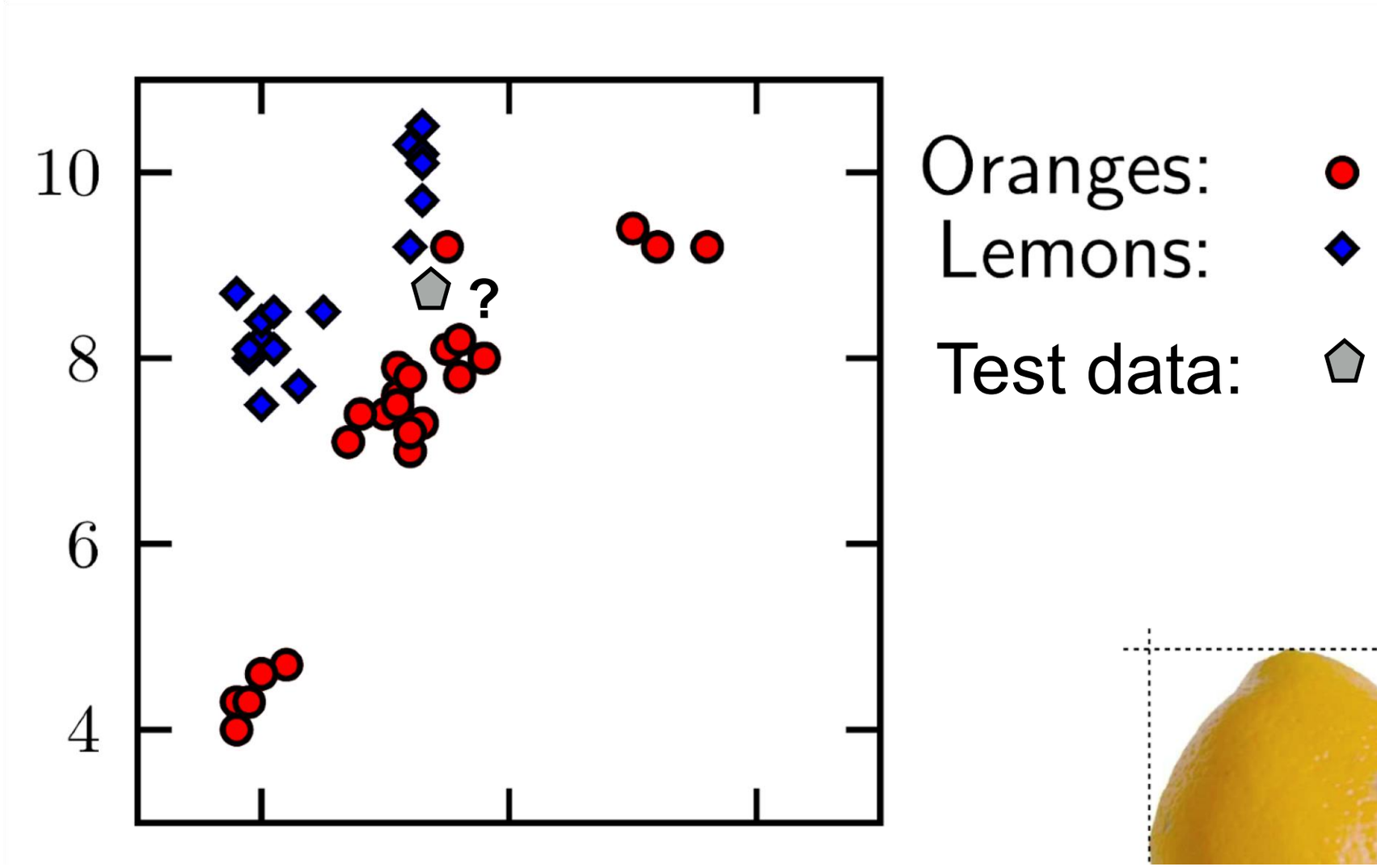
Test data:

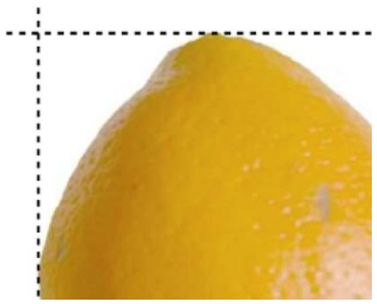


$y_{new} \leftarrow y_{i^*}$

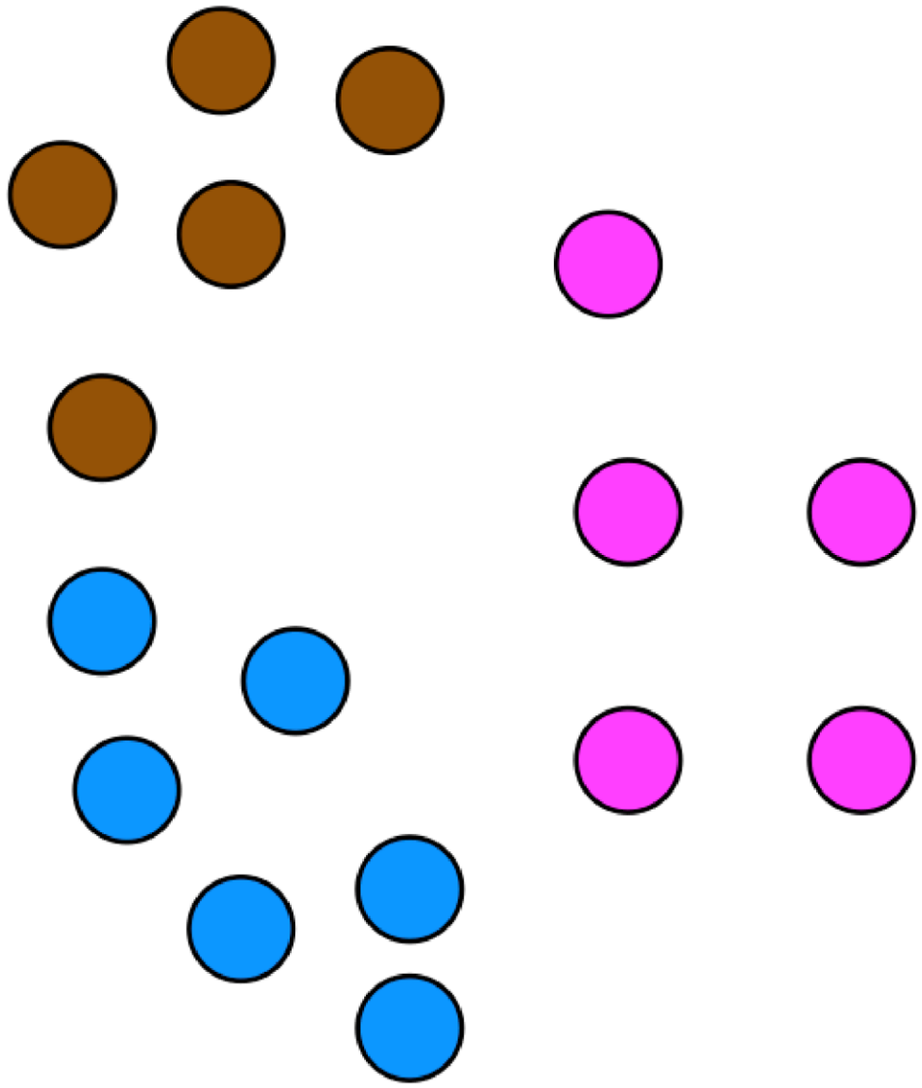
$$i^* = \underset{i}{\operatorname{argmin}}\ dist(\mathbf{x}_i, \mathbf{x}_{new})$$

1-Nearest Neighbour



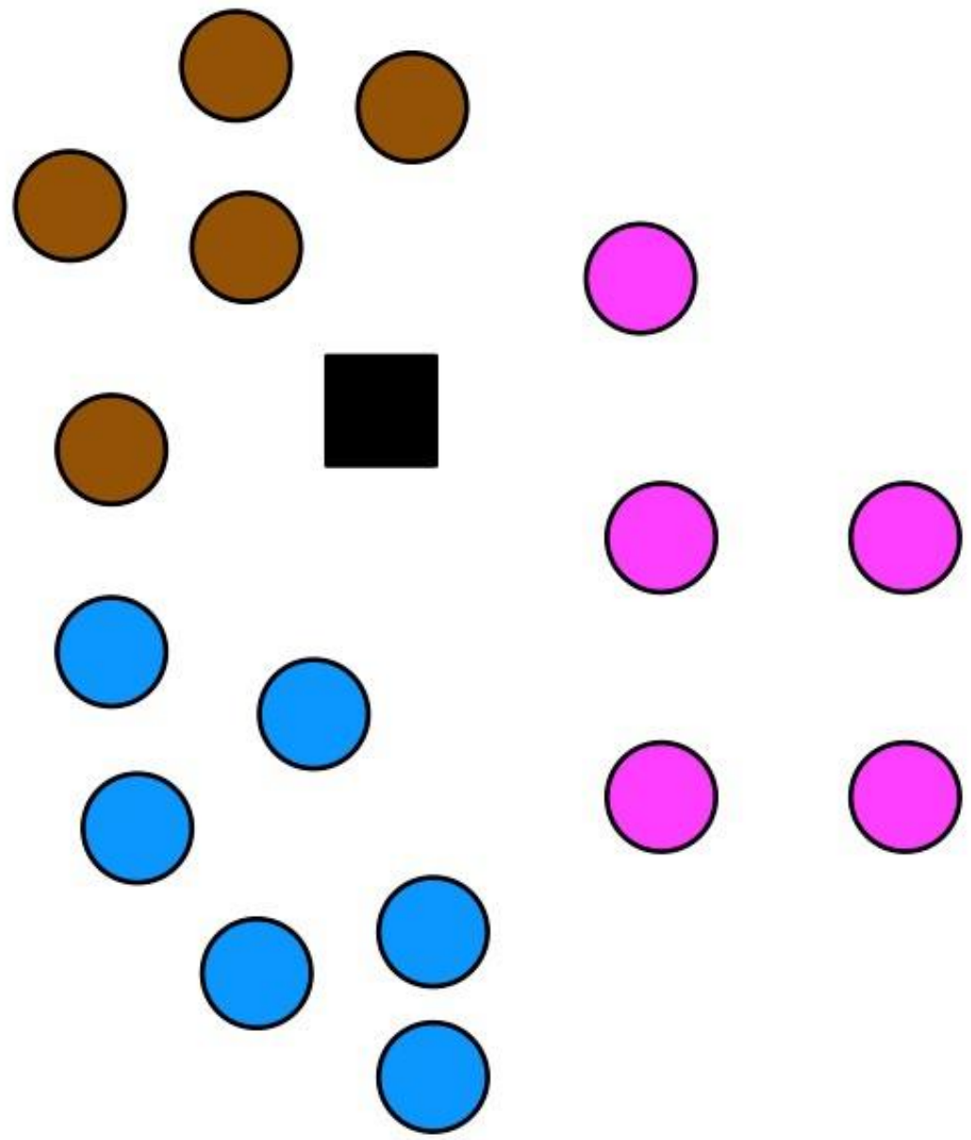


K-Nearest Neighbour



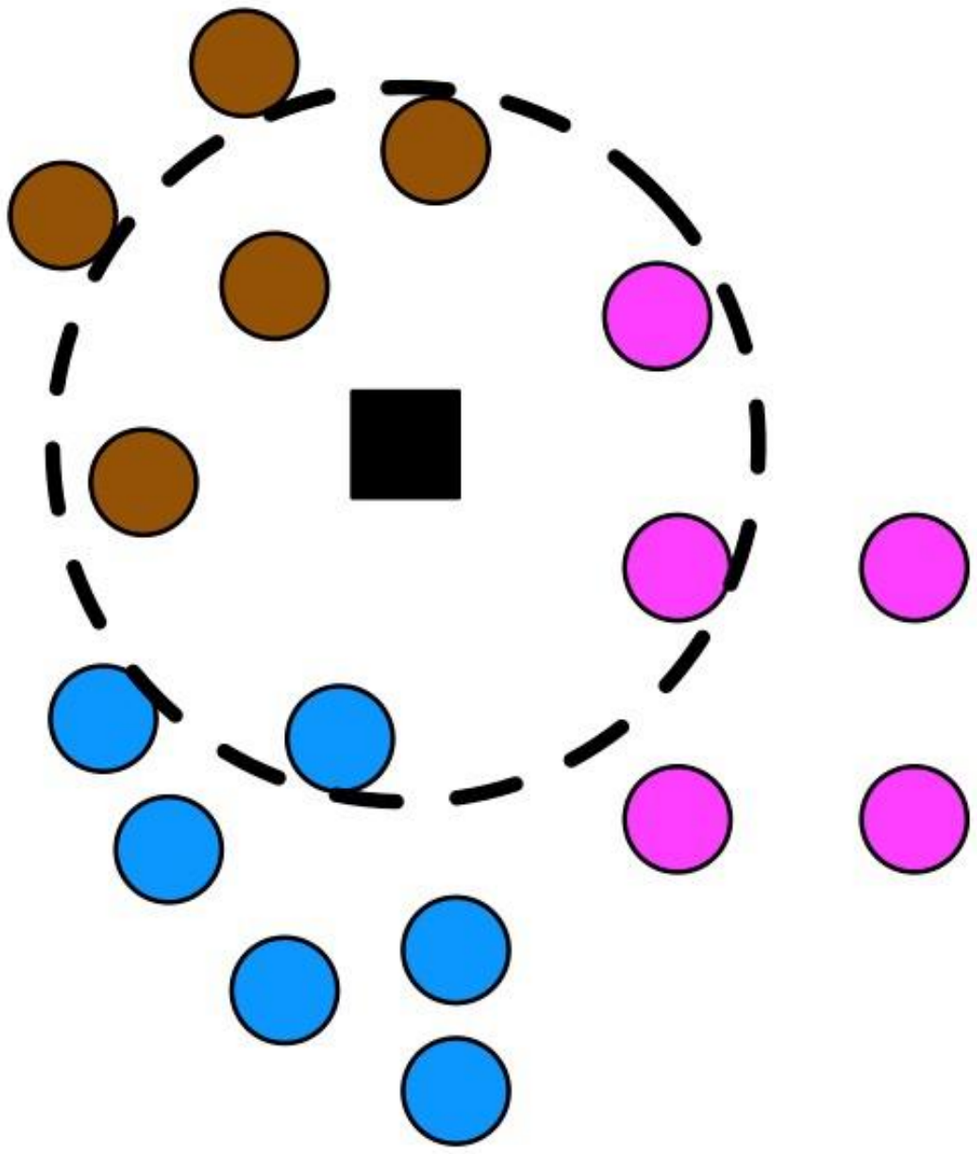
Training data from 3 classes.

K-Nearest Neighbour



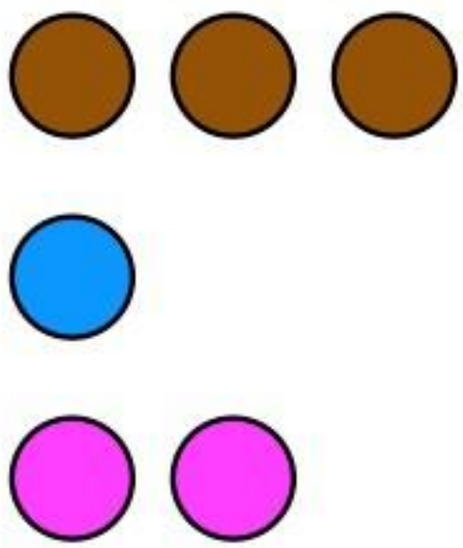
Test point.

K-Nearest Neighbour



Find $K = 6$ nearest neighbours.

K-Nearest Neighbour



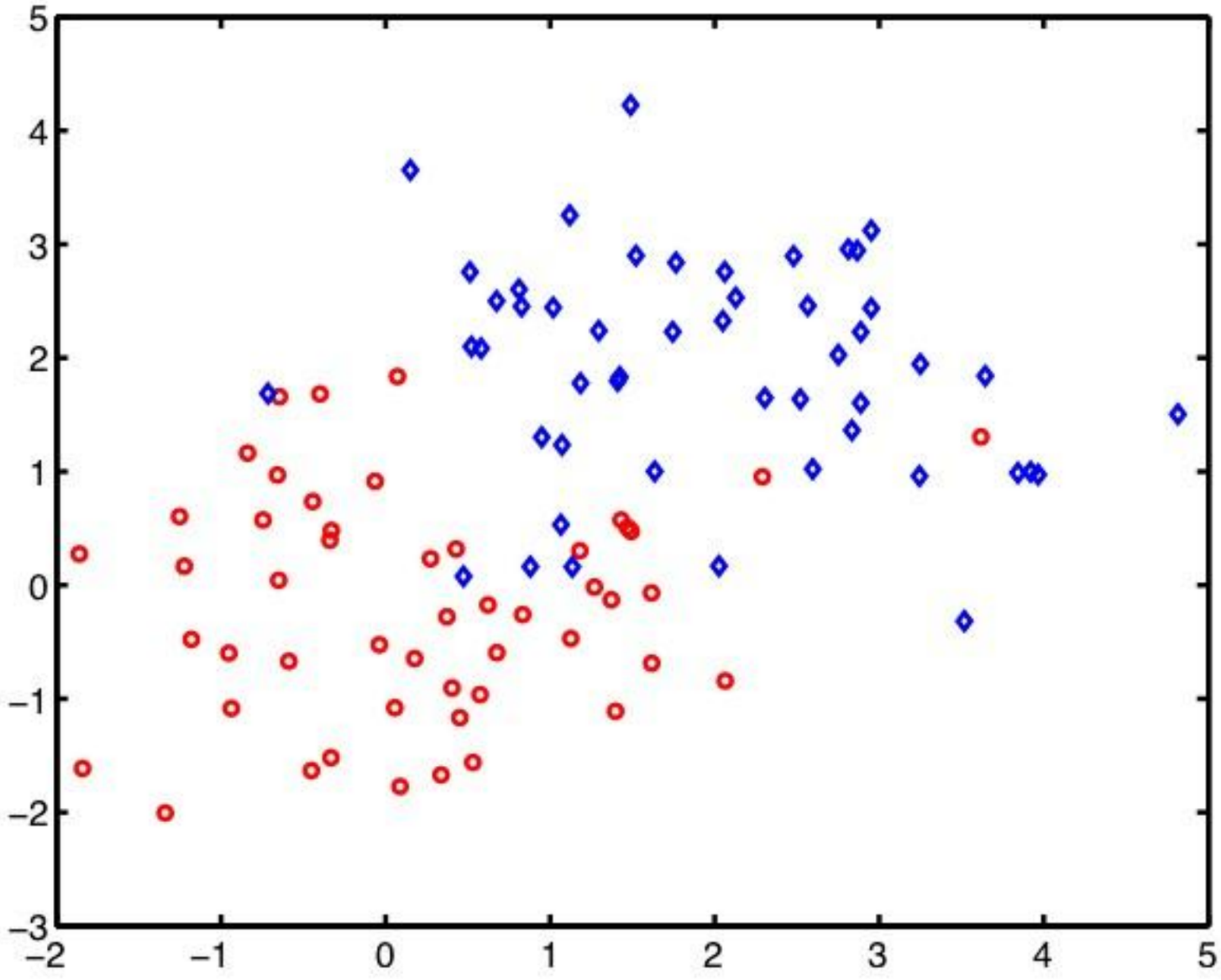
3 from class 1

1 from class 2

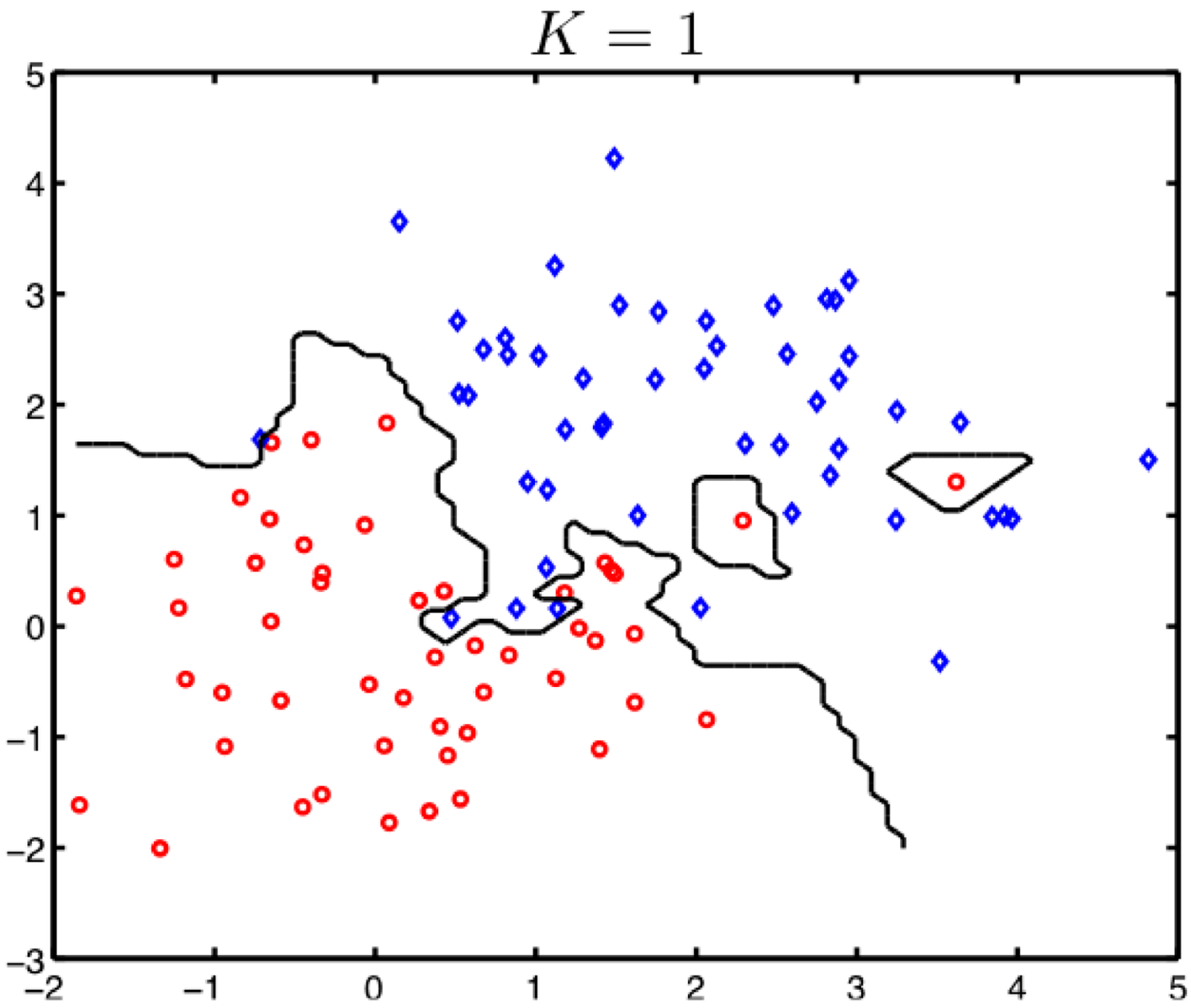
2 from class 3

Class one has most votes – classify $\mathbf{x}_{new}$ as belonging to class 1.

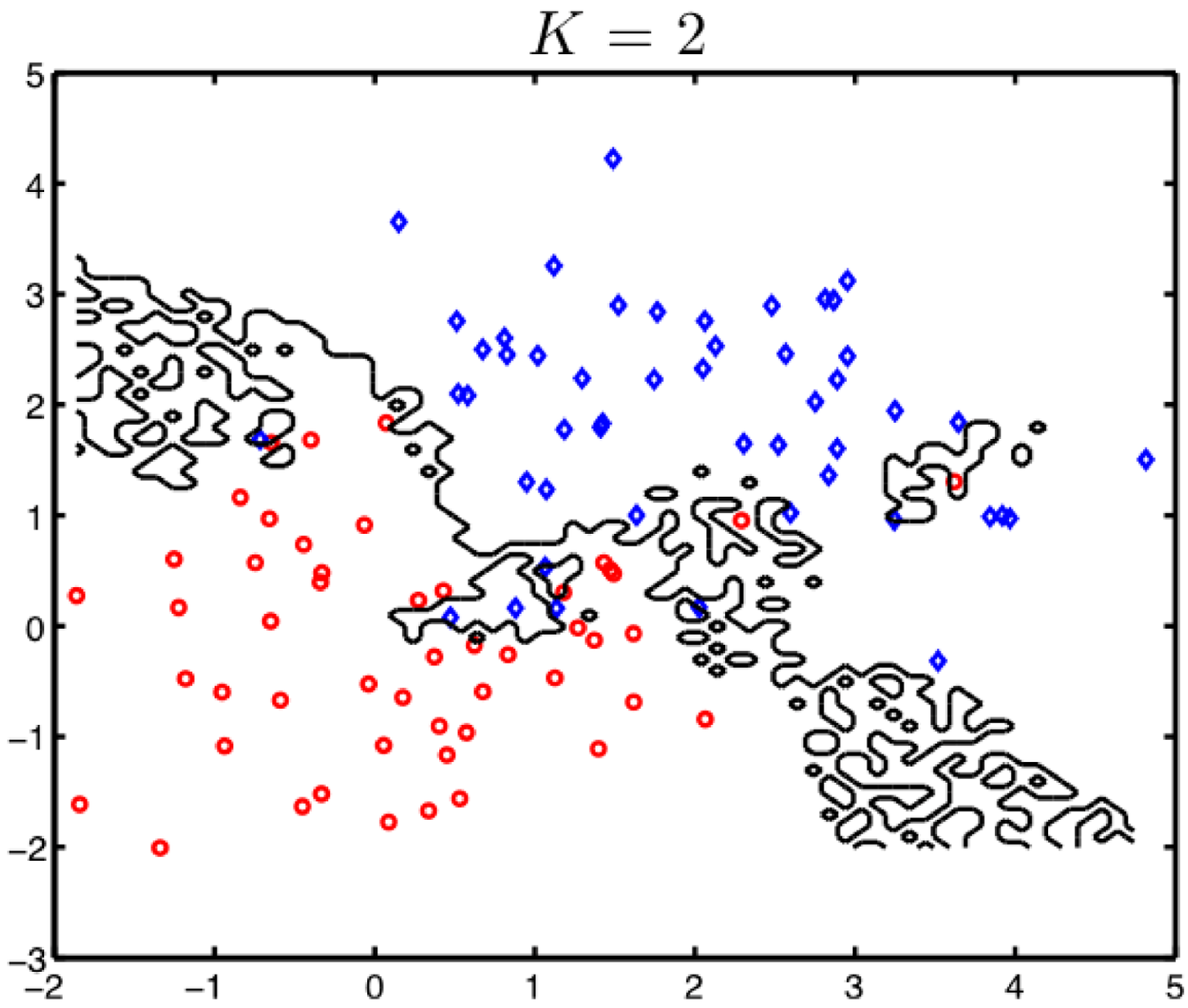
K-Nearest Neighbour



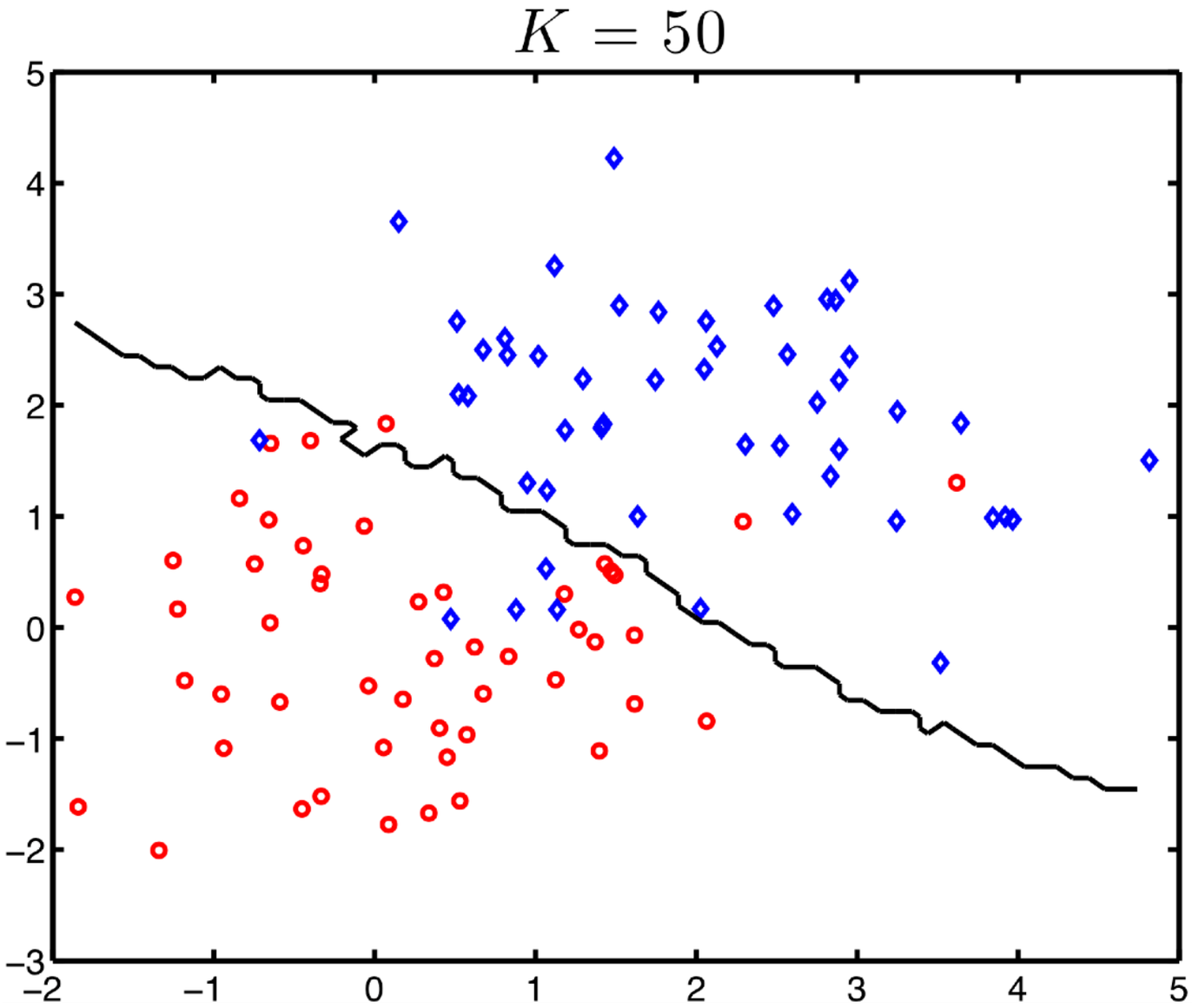
K-Nearest Neighbour



K-Nearest Neighbour



K-Nearest Neighbour



K-Nearest Neighbour

- **What is the training process?** Are we learning any parameters?
- 'Training' in K-NN = 'Memorizing' training data

- **How do we compute the distance between samples?**
- Any distance metric should work. Squared *L2* norm is common for real-valued features.

$$||\mathbf{x}_1 - \mathbf{x}_{new}||_2^2 = \sum_i \left(x_1(i) - x_{new}(i)\right)^2$$

- Choice of distance metric may change results (but not much).

K-Nearest Neighbour

- Class imbalance
  - As K increases, small classes will disappear!
  - Imagine we had only 5 training objects for class 1 and 100 for class 2.
  - For $K \geq 11$, class 2 will **always** win!

- How do we choose K?
  - Right value of K will depend on data.
  - Cross-validation!

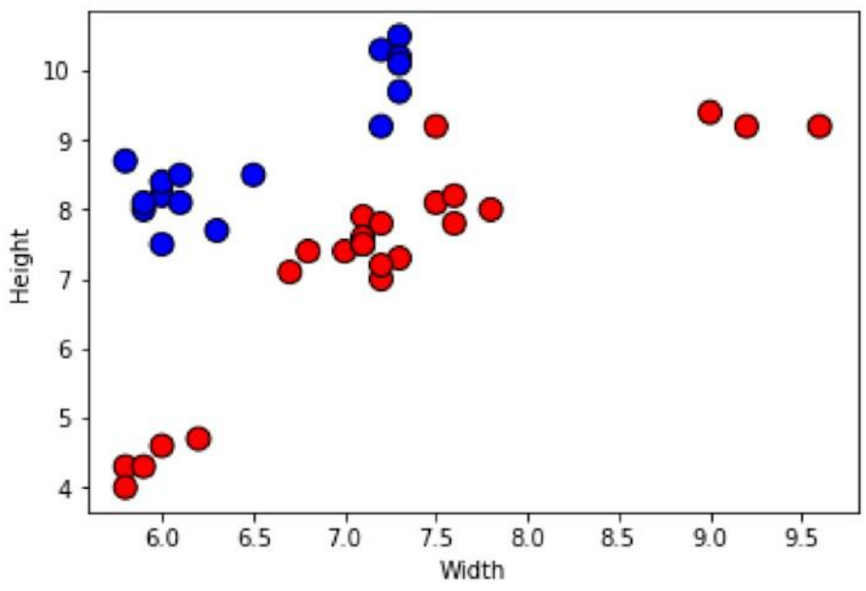
K-NN (orange & lemon example)

In [2]:
```
import numpy as np
%matplotlib inline
import pylab as plt
from matplotlib.colors import ListedColormap

# Create color maps
cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])
cmap_bold = ListedColormap(['#FF0000', '#00FF00', '#0000FF'])

data = np.loadtxt('orange_lemon.txt', delimiter=',') # load fruit data
X = data[:,1:3]
t = data[:,0]
plt.scatter(X[:, 0], X[:, 1], c=t, cmap=cmap_bold, edgecolor='k', s=100)
plt.xlabel('Width')
plt.ylabel('Height')
```

Out[2]:
```
Text(0, 0.5, 'Height')
```

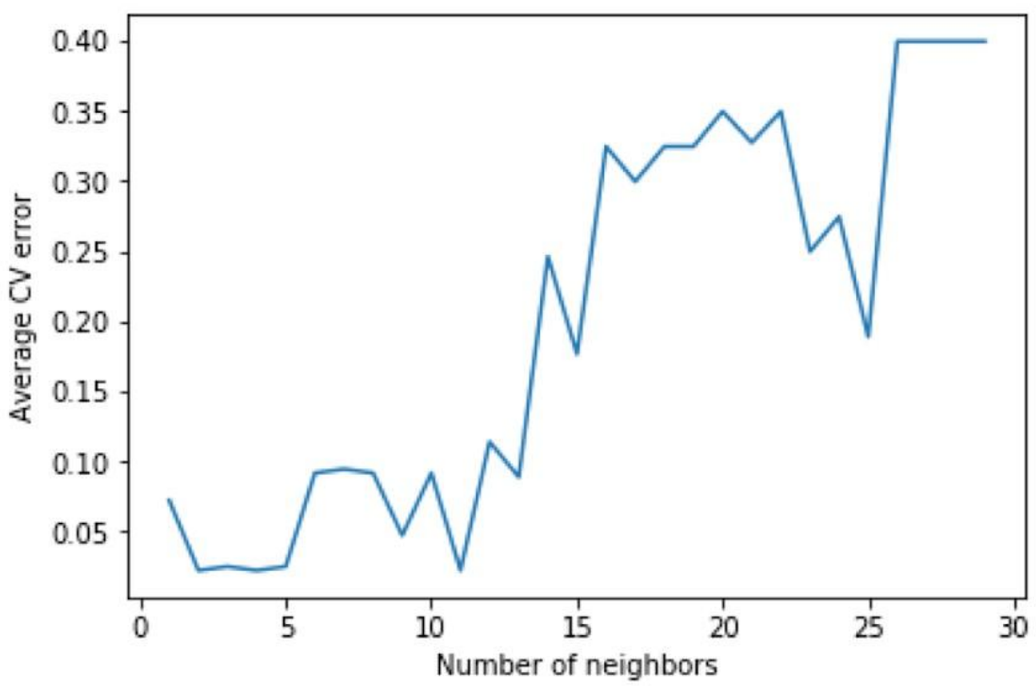


K-NN (orange & lemon example)

In [17]:
```
cv_scores = []
for i in range(1,30,1):
    knn_cv = KNeighborsClassifier(n_neighbors=i)
    cv_scores.append(1-np.mean(cross_val_score(knn_cv, X, t, cv=5)))

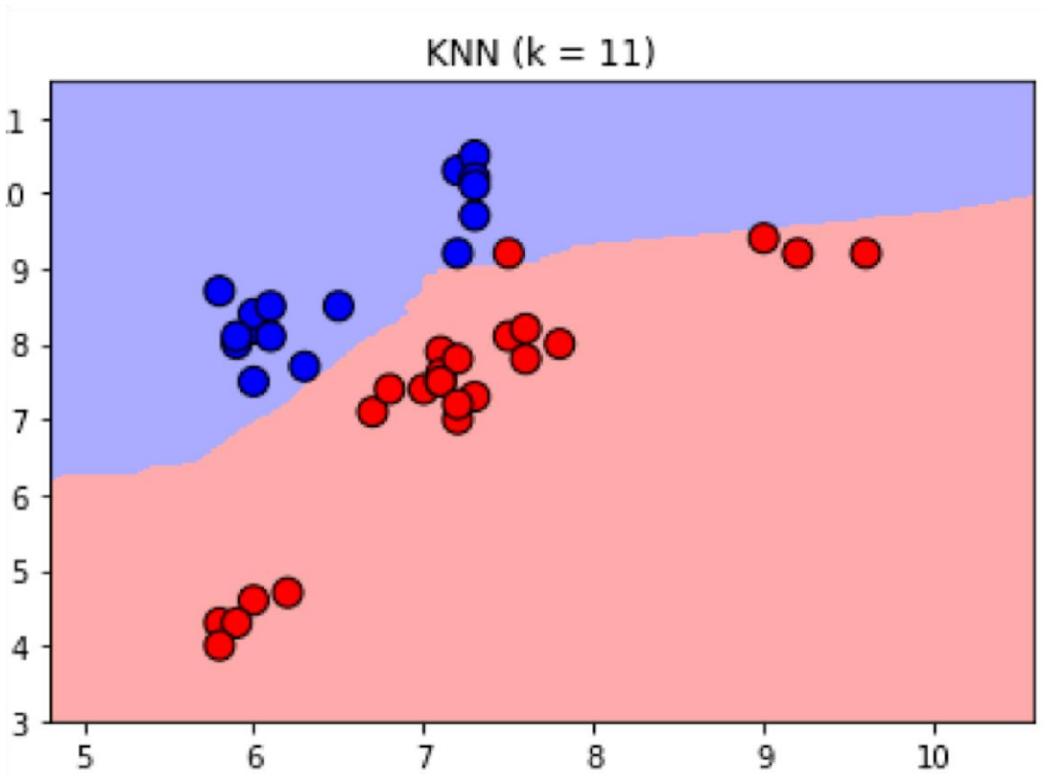
plt.plot(np.arange(1,30,1),cv_scores)
plt.xlabel('Number of neighbors')
plt.ylabel('Average CV error')
print(np.min(cv_scores))
```

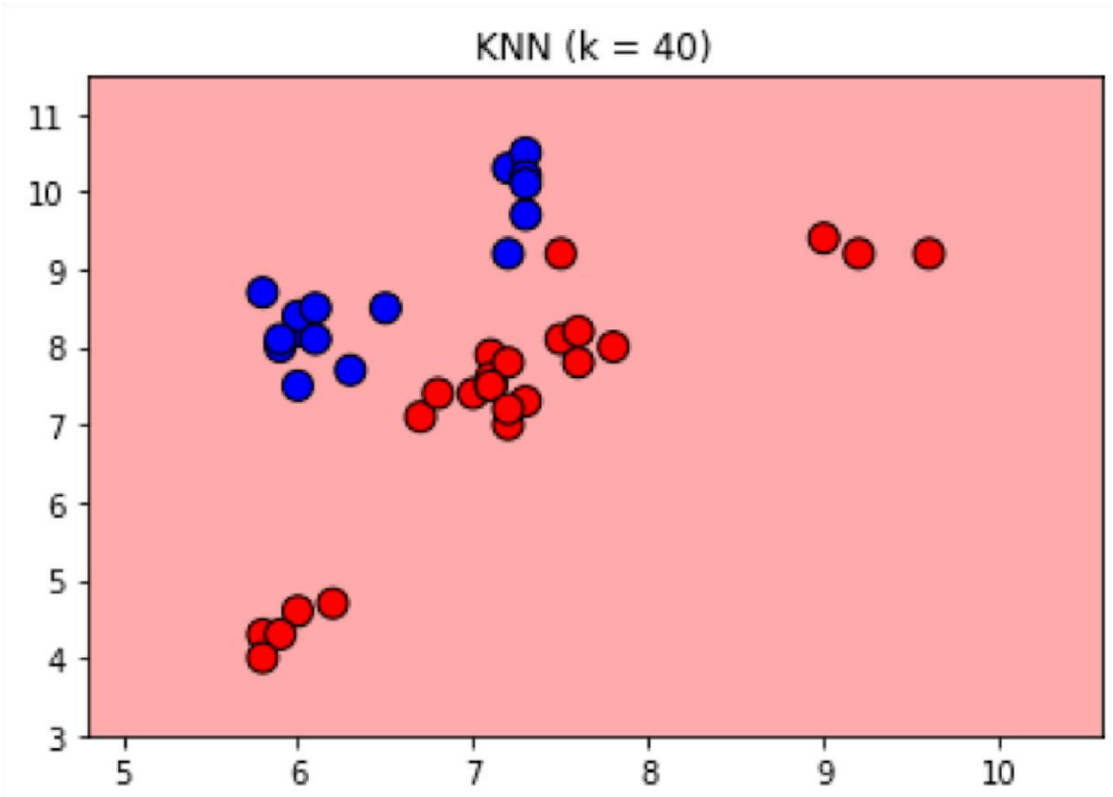
```
0.02222222222222143
```



5 fold cross validation to choose K

K-NN (orange & lemon example)





K-NN summary

Simple

- Only 1 parameter to tune
- simple to implement
- Fast training (rather no training)

...but **inefficient**

- Inference time may be large if *N* is large - **Not ideal**
- Large memory requirement
- Vulnerability to class imbalance
- Sensitivity to noise at small K